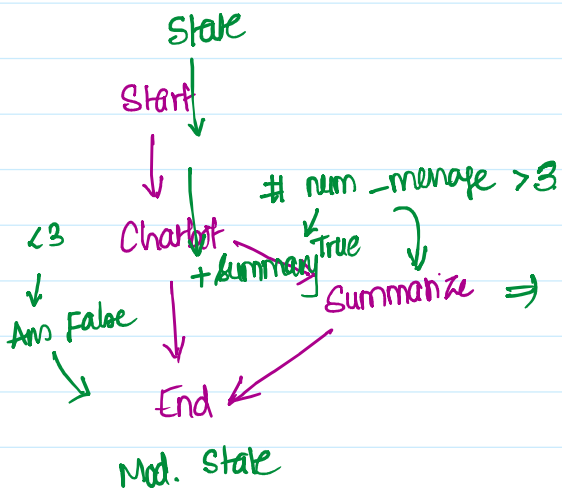
# AWS Bedrock RAG Application

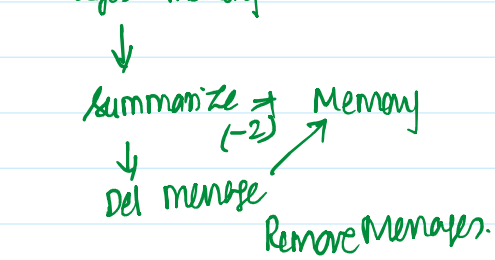
Tuesday, March 31, 2026 6:04 PM

**Recap :**

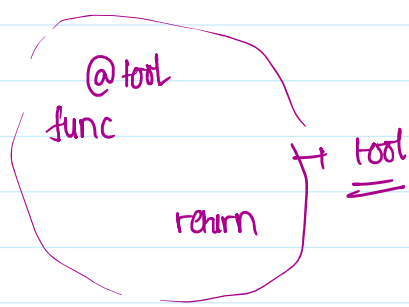
1) Summarize memory → Content to LLM.
2) Add Tool to LLM [ long term Mem ]
3) Add HITL - Lang Graph

Summarization




Add Tool
→ LLM.




1) tool node ⇒ ToolNode ( add the list of single ) @ Creating node

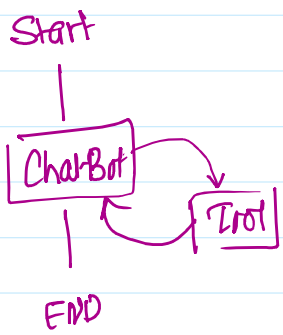
tools =

add_node ( 'tools', tools ).

2) add_conditional_edge ( 'chatbot', tool_condition, { 'tools' : 'tools' END : END }

tool_condition

Start
|




{ tools : tool
END : END }

3) Human in the Loop.

Start

ChatBot

End

Tool

@tool
get_user_feedback()
doc "'
Bef returning o/p to user call get-user feed
"'

@tool
def get_user_feedback()

(interrupt

Pause { question : _____ ;
response ! }.

User needs to verify the output & respond it

resp. = Command.{
response = 'Good resp'
}

graph. invoke( resp.
Config.
)

interrupt (Pause)

Command (Resume) ⇒ Resp → LLM. → resp - good

Modify feedback

LLM ⟷ Tool

print o/p

Implement / Build a simple RAG application

New data / proprietary data → More

- Amazon Bedrock

1) Indexing
- Doc load ✓ ✓ S3 bucket
- Text split ✓ doc → Chunks ✗
- Create Emb chunks → emb ✗
- add Vector emb → vec db ✗

Retrieval ✗

get the relevant info

Generation ✓

Available in Vector DB

To the query → emb → Vector DB → apply a sim search

↓

Similar top-K search result

User query + Text

LLM

↓

Better response

1) ✓ Load doc - S3 bucket

2) Create a Knowledge Base

doc → Chunks → emb

↓

Storing in DB. ✓

User ask a query

content similar to query

+

3) Generate the response

Query → Response

Applicn AWS Bedrock

↓

Id KB

Query → Response

Id KB
↓
Streamlit
↓
AWS EC2

Knowledge base id is available
↓
Knowledge base id
& Model name, Model ARN

SDK
↓
Boto3.
↘ Amazon Aws.

Stream appli

retrieve and generate
↓
KB ID.
& Model ARN
→ Bedrock
↓
Query.